REAL-TIME THREAT & ANOMALY DETECTION SYSTEM

A PROJECT REPORT

Submitted in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF COMPUTER APPLICATIONS

by

MAYANK SINGH
BHASHKAR PATHAK
RAJ FARSWAN

Under the supervision of

Ms. Nidhi Joshi
Assistant Professor, Department of School of Computing

SCHOOL OF COMPUTING
GRAPHIC ERA HILL UNIVERSITY
CLEMENTOWN, DEHRADUN-248002, UTTARAKHAND, INDIA

October 2026

REAL-TIME THREAT & ANOMALY DETECTION SYSTEM

PROJECT REPORT

Mayank Singh & Team

Second-phase implementation report for an explainable Application Detection and Response prototype using Java Spring Boot, Weka machine learning, relational persistence, and a browser-based SOC dashboard.

CANDIDATE'S DECLARATION

We certify that the work presented in this Project Report entitled **?Real-Time Threat & Anomaly Detection System?**, submitted in partial fulfilment of the requirements for the award of the Degree of Bachelor of Computer Applications at the School of Computing, Graphic Era Hill University, Dehradun, is an authentic record of the work carried out by our team under the supervision of Ms. Nidhi Joshi. It has not been submitted for another degree.

Student	Roll Number	Signature
Mayank Singh	2421376	_____
Bhashkar Pathak	2421199	_____
Raj Farswan	2421476	_____

Date: _____

Signature of Mentor: _____
Ms. Nidhi Joshi

ABSTRACT

This project presents a real-time Threat and Anomaly Detection System implemented as an Application Detection and Response prototype. A Java Spring Boot backend intercepts HTTP traffic through a servlet filter, performs a pre-execution IP blocklist check, inspects query parameters and headers for SQL injection, cross-site scripting, and path traversal signatures, and stores request telemetry in a relational database. Telemetry from each client IP is aggregated over a 60-second sliding window into eight behavioural features: request count, failed authentication count, 404 count, unique endpoints, error ratio, average response time, average payload size, and signature hits. A Weka RandomForest classifier predicts Normal, BruteForce, EndpointScan, AppDoS, PayloadAttack, or BehavioralAnomaly. Rule weights and classifier probability are combined into a 0?100 risk score with explainable severity levels. Critical scores trigger a temporary automatic IP block, while SOC users can triage alerts, adjust sensitivity, block or unblock IPs, and export telemetry. The project includes a browser dashboard and a safe attack simulator. The prototype demonstrates the complete detection cycle from observation and classification to explanation, response, triage, and audit logging.

Keywords: cybersecurity, anomaly detection, Weka, RandomForest, Spring Boot, WAF, SOC, risk scoring

ACKNOWLEDGEMENTS

We express our sincere gratitude to Ms. Nidhi Joshi, Assistant Professor, Department of School of Computing, Graphic Era Hill University, for her guidance and feedback. We thank the School of Computing for the academic environment and resources. We also acknowledge the contributions of all team members in requirements analysis, backend development, machine-learning integration, frontend demonstration, testing, and documentation.

Mayank Singh
Bhashkar Pathak
Raj Farswan

CONTENTS

Section	Title
Chapter 1	Introduction and Problem Statement
Chapter 2	Literature Review and Background
Chapter 3	System Analysis and Requirements
Chapter 4	Proposed System and Design
Chapter 5	Implementation and Machine Learning Pipeline
Chapter 6	Testing, Results and Discussion
Chapter 7	Conclusion and Future Work
References	
Appendix A	Demonstration Commands

LIST OF ABBREVIATIONS

ADR	Application Detection and Response
API	Application Programming Interface
ARFF	Attribute-Relation File Format
CORS	Cross-Origin Resource Sharing
HTTP	Hypertext Transfer Protocol
JPA	Java Persistence API
ML	Machine Learning
RBAC	Role-Based Access Control
SOC	Security Operations Centre
WAF	Web Application Firewall

CHAPTER 1
INTRODUCTION AND PROBLEM STATEMENT

1.1 Introduction

Web applications generate large volumes of request and response telemetry. Manual inspection is too slow, while purely rule-based detection may miss slow or previously unseen behaviour. This project combines deterministic rules, behavioural features, Weka classification, and an SOC dashboard.

1.2 Motivation

The aim is to provide a small, explainable, and demonstrable security system that identifies suspicious traffic, explains the reason, persists evidence, supports analyst decisions, and applies containment for critical events.

1.3 Problem Statement

Design and implement a backend service that intercepts HTTP requests, rejects blocked clients before execution, detects common Layer 7 attack signatures, aggregates features per client IP, classifies activity using Weka, calculates risk, generates explainable alerts, supports SOC triage, and records administrative actions.

1.4 Objectives and Scope

• Capture client IP, user ID, endpoint, method, status, timing, payload size, user agent, signature count, and timestamp.

• Detect SQL injection, XSS, and path traversal before application processing.

• Train and use Weka on eight exact behavioural features.

• Provide risk scoring, sensitivity, automated blocking, RBAC, audit logging, CSV export, and safe demonstrations.

CHAPTER 2
LITERATURE REVIEW AND BACKGROUND

2.1 Rule-Based Detection

Signature detection is fast, deterministic, and easy to audit. It is useful for known patterns such as SQL injection, XSS, traversal, repeated authentication failures, and excessive latency. Its limitation is that attackers can vary syntax or use behaviour outside known signatures.

2.2 Behavioural Anomaly Detection

Behavioural detection represents activity numerically. Request volume, endpoint diversity, status-code distributions, payload sizes, and response times provide signals for brute force, scanning, DoS, payload abuse, and stealthy anomalies.

2.3 Explainable Security Analytics

SOC decisions require evidence. Each alert therefore stores the predicted class, confidence, relevant feature values, rule signal, risk score, and a human-readable reason.

2.4 Proposal Alignment

The first-phase proposal identified cybersecurity, AI/ML, Java Spring Boot, Weka/Tribuo, PostgreSQL, and a SOC dashboard. The second-phase implementation delivers the core application using Spring Boot, Weka RandomForest, JPA, H2/PostgreSQL compatibility, and a browser dashboard. Kafka/WebSockets, React/Tailwind, and larger unsupervised models are future enhancements.

CHAPTER 3
SYSTEM ANALYSIS AND REQUIREMENTS

3.1 Functional Requirements

ID	Requirement	Implementation
FR-01	Intercept HTTP requests	SecurityTelemetryFilter
FR-02	Check active IP blocks	BlockedIpRepository and filter
FR-03	Store telemetry	RequestLog entity
FR-04	Aggregate 60-second features	TelemetryFeatureAggregator
FR-05	Classify with Weka	WekaAnomalyService
FR-06	Create explainable alerts	SecurityAlert
FR-07	Contain critical IPs	15-minute block
FR-08	SOC operations	SocOperationsController
FR-09	Safe demonstrations	DemoAttackController

3.2 Security Requirements

• Use JPA parameterized operations instead of SQL concatenation.

• Separate SOC_VIEWER and SOC_ADMIN privileges.

• Record administrative actions in admin_audit_logs.

• Reject active blocked clients with HTTP 403.

• Keep simulated attacks local and synthetic.

3.3 Technology Stack

Layer	Technology
Backend	Java 21, Spring Boot 3.5, Spring MVC
Persistence	Spring Data JPA, H2 demo database, PostgreSQL-compatible schema
Machine learning	Weka 3.8.6 RandomForest
Frontend	HTML, CSS, browser JavaScript
Build	Maven wrapper
Data	ARFF training dataset and CSV export

CHAPTER 4
PROPOSED SYSTEM AND DESIGN

4.1 Architecture

The frontend and APIs generate HTTP traffic. The servlet filter is the first security boundary: it checks the blocklist, inspects parameters and headers, allows normal processing, and records the completed request. The feature aggregator queries the last 60 seconds. The detection service combines rules and Weka output. Alerts and blocks are persisted for SOC use.

Component	Responsibility
SecurityTelemetryFilter	Block check, signatures, timing and telemetry
TelemetryFeatureAggregator	Eight-feature rolling-window calculation
WekaAnomalyService	Training, inference, risk, and reason
SecurityAlert	Alert persistence and triage
SocOperationsController	Stats, triage, block/unblock, sensitivity, export
DemoAttackController	Safe bruteforce, scan, DoS, SQLi simulations

4.2 Request Processing Flow

HTTP request ? active blocklist check ? signature inspection ? controller execution ? telemetry persistence ? 60-second aggregation ? rules and Weka ? risk and severity ? alert ? automatic block when risk exceeds 90 ? SOC triage and audit.

4.3 Database Design

The schema contains request_logs, blocked_ips, security_alerts, and admin_audit_logs. Request logs are indexed by client IP and timestamp to support rolling-window aggregation.

CHAPTER 5
IMPLEMENTATION AND MACHINE LEARNING PIPELINE

5.1 Telemetry Interceptor

SecurityTelemetryFilter extends OncePerRequestFilter. It obtains the client address, checks active temporary or permanent blocks, inspects decoded servlet parameters and headers for SQL injection, XSS, and traversal patterns, wraps the request to measure payload size, and records status code and response time.

5.2 Feature Aggregation

Feature	Meaning
req_count_60s	Total requests in 60 seconds
failed_auth_count	HTTP 401 and 403 count
error_404_count	HTTP 404 count
unique_endpoints	Distinct paths
error_rate_ratio	(4xx + 5xx) / total
avg_response_time_ms	Mean processing time
avg_payload_bytes	Mean payload size
signature_hit_count	Suspicious signature matches

5.3 Weka Model

cyber_traffic.arff defines the eight numeric attributes and threat_class with Normal, BruteForce, EndpointScan, AppDoS, PayloadAttack, and BehavioralAnomaly. At startup, WekaAnomalyService loads the data, builds a RandomForest, and writes cyber_traffic.model. A DenseInstance uses the same attribute order for distributionForInstance().

5.4 Risk and Explainability

Classifier confidence is combined with rule-derived signals and clamped to 0?100. Severity is Normal for 0?30, Low for 31?60, High for 61?84, and Critical for 85?100. Sensitivity can be changed by SOC_ADMIN.

CHAPTER 6
TESTING, RESULTS AND DISCUSSION

6.1 Test Scenarios

Scenario	Generated behaviour	Observed result
Brute force	30 requests and 30 failed auth responses	BruteForce, risk 100
Endpoint scan	45 distinct paths and 45 404 responses	EndpointScan, risk 100
Application DoS	20 high-latency requests	High-risk alert
SQL injection	Five requests with signature hit	PayloadAttack, risk 90
Encoded SQLi	Decoded OR 1=1 query	PayloadAttack, risk 85
Manual block	Admin blocks test IP	HTTP 403
RBAC	Viewer attempts sensitivity update	HTTP 403; admin succeeds

6.2 Observed Results

The packaged Spring Boot application started successfully with H2, JPA, and Weka. The SOC statistics endpoint returned HTTP 200. All four safe simulations returned JSON with risk and alert identifiers. The browser dashboard loaded on port 8000 and the Threat Lab buttons displayed generated request counts and risk results. A browser CORS issue found during testing was corrected for the localhost origins.

6.3 Limitations

• H2 is for evaluation; PostgreSQL is recommended for deployment.

• SOC role headers must be replaced with JWT/OIDC authentication.

• The ARFF dataset is small and curated for demonstration.

• The signature scanner is lightweight, not a commercial WAF.

• The simulator creates synthetic local telemetry only.

CHAPTER 7
CONCLUSION AND FUTURE WORK

7.1 Conclusion

The project delivers a working, explainable real-time threat detection prototype. It combines fast rules with Weka classification, persists telemetry and alerts, provides a browser SOC dashboard, supports analyst triage, records administrative actions, and applies automatic temporary containment for critical risk. The operational loop is Observe ? Aggregate ? Classify ? Explain ? Alert ? Block ? Triage ? Audit.

7.2 Future Work

• Replace demonstration headers with Spring Security and JWT/OIDC.

• Add Kafka or WebSocket streaming and live alert push.

• Use a larger sanitized dataset and compare RandomForest, J48, Isolation Forest, and calibrated models.

• Add rate limiting, HTTPS, secrets management, PostgreSQL migrations, and CI/CD security testing.

• Add charts for rolling risk, endpoint heatmaps, confusion matrices, and model drift.

REFERENCES

[1] Weka 3.8 Documentation, Waikato Machine Learning Project, https://www.cs.waikato.ac.nz/ml/weka/

[2] Spring Boot Reference Documentation, https://docs.spring.io/spring-boot/docs/current/reference/html/

[3] OWASP Foundation, OWASP Top 10 Web Application Security Risks, https://owasp.org/www-project-top-ten/

[4] OWASP Foundation, OWASP Application Security Verification Standard, https://owasp.org/www-project-application-security-verification-standard/

[5] Spring Data JPA Reference Documentation, https://docs.spring.io/spring-data/jpa/reference/

APPENDIX A
DEMONSTRATION COMMANDS

Start backend

```
cd "C:\Users\singh\Downloads\threat and anomanly\backend"
.\mvnw.cmd spring-boot:run
```

Start frontend

```
cd "C:\Users\singh\Downloads\threat and anomanly\frontend"
python -m http.server 8000
```

Run SQL injection demo

```
curl.exe -X POST "http://localhost:8080/api/demo/simulate-attack?type=sqli"
```

Read alerts

```
curl.exe http://localhost:8080/api/soc/alerts
```

Manual admin block

```
curl.exe -X POST http://localhost:8080/api/soc/block -H "Content-Type: application/json" -H
"X-SOC-Role: SOC_ADMIN" -H "X-SOC-User: admin@example.com" -d
"{"ip":"203.0.113.10","reason":"Demo containment","minutes":15,"permanent":false}"
```

The project source includes the ARFF dataset, Java services/controllers/entities, application configuration, frontend dashboard, and database schema.

APPENDIX B
FOUR-MEMBER PROJECT TASK DIVISION

The project work is divided among four members so that requirements analysis, backend implementation, machine learning, frontend development, security testing, and documentation are independently owned while remaining integrated through the team leader.

Member	Academic details	Assigned responsibility	Deliverables
1. Mayank Singh Team Leader	Roll No: 2421376 BCA B1	Project coordination and backend integration; Spring Boot application structure; REST API integration; final system integration; demonstration and report coordination.	ThreatGuardApplication; controller integration; end-to-end flow; final presentation; consolidated report
2. Bhashkar Pathak	Roll No: 2421199 BCA B1	Frontend SOC dashboard; dashboard layout and styling; live statistics, alerts, filters, triage controls, and Threat Lab buttons.	Frontend pages; JavaScript API calls; SOC event views; simulator UI; browser demonstration
3. Raj Farswan	Roll No: 2421476 BCA B1	Machine-learning and Weka pipeline; ARFF dataset; feature schema; RandomForest training; probability inference; risk scoring and explainability.	cyber_traffic.arff; WekaAnomalyService; model evaluation; threat-class explanations
4. Pawan Panday	Roll No: 2421434 BCA B1	Database, telemetry security, testing, and deployment support; request-log schema; blocklist and audit verification; API smoke tests; documentation evidence.	schema.sql; telemetry/blocklist validation; RBAC and 403 tests; CSV/export checks; deployment and test notes

Integration protocol: Each member commits work to the shared project workspace. The team leader reviews interfaces and combines the frontend, backend, database, and Weka components. Joint testing covers attack simulation, alert creation, automatic blocking, RBAC, audit logging, CSV export, and browser operation.

Team signatures: Mayank Singh _____ Bhashkar Pathak _____

Raj Farswan _____ Pawan Panday _____